

Chigma: A Secure, Multi-Agent Orchestrated Autonomy and Visual Inspection System for Defense Surveillance

Naman Gautam

*Department of Computer Science and Engineering
Indian Institute of Information Technology Vadodara
Gandhinagar, Gujarat, India
namangautam172@gmail.com*

Abstract—This paper presents Chigma, an industrial-grade tactical AI and flight control platform designed for secure autonomous reconnaissance and defect detection. Chigma integrates a custom FD-YOLO11 defect detection neural network, a real-time Visual Odometry / SLAM pose recovery engine, a Hierarchical Multi-Agent Flight safety validator, and a MongoDB-backed Role-Based Access Control (RBAC) security auditing framework. We demonstrate the efficacy of this multi-layered architecture in ensuring flight stability under wind turbulence, enforcing geofence ceilings, and validating metal plating defects in real-time.

Index Terms—Autonomy, Multi-Agent Systems, Visual SLAM, Defect Detection, Role-Based Access Control, Edge AI, Tactical Surveillance.

I. INTRODUCTION

Modern military and industrial defense frameworks demand robust, high-fidelity autonomous agents capable of performing structural integrity inspection and surveillance in GPS-denied tactical environments. Operations such as inspecting armored vehicles, monitoring military fortifications, and surveying border lines require unmanned aerial vehicles (UAVs) equipped with state-of-the-art computer vision models. However, deploying vision-guided drones introduces two critical vulnerabilities: (a) the dependency on unreliable GPS networks for localized navigation, and (b) the absence of secure, role-restricted control channels, leaving the autopilot exposed to unauthorized flight commands or sensor spoofing.

To mitigate these security and perception challenges, we introduce **Chigma**, a defense-grade, multi-agent orchestrated autonomy and visual inspection platform. Unlike traditional autopilots (e.g., PX4 or ArduPilot) that operate on simple, unchecked command queues, Chigma forces all incoming instructions through a Hierarchical Multi-Agent verification framework. Natural language commands are parsed by a Planner Agent, mapped to physical coordinates by an Executor Agent, and validated against hard geofence and battery safety limits by a Safety Agent before being sent to the flight controller.

Furthermore, Chigma features a custom visual defect classification engine based on the ‘FD-YOLO11’ architecture,

which incorporates DySample upsampling and Feature Pyramid Networks (FSPPF) to detect cracks and anomalies on metal plating. In GPS-denied environments, Chigma leverages a monocular visual SLAM odometry system using ORB feature descriptor matching and Essential Matrix pose recovery to track position offsets. Finally, to secure access to these critical capabilities, the API enforces role-based security layers (Commander, Operator, Observer) and maintains a write-ahead audit trail on a MongoDB cluster.

Our primary contributions are summarized as follows:

- 1) We design and implement a security-hardened FastAPI control interface that enforces JWT authentication and RBAC, logging all strategic decisions and access attempts to a MongoDB audit trail.
- 2) We formulate a Hierarchical Multi-Agent system that translates natural language commands into verified, geofence-compliant autopilot coordinates.
- 3) We integrate a real-time Visual SLAM camera pose tracker and a custom PyTorch YOLO defect detection network that evaluates anomalies.
- 4) We present exhaustive evaluations demonstrating that the multi-agent system successfully blocks out-of-bound flights and handles wind gusts up to 15 m/s.

II. RELATED WORK

Autonomous inspection platforms represent a confluence of deep learning, computer vision, robotics, and cyber security. This section surveys existing research across these domains.

A. Object Detection in Industrial Inspection

Defect identification has transitioned from manual inspector evaluations to convolutional neural networks (CNNs). Ultralytics’ YOLO models are widely used for real-time edge processing due to their balance of speed and precision. However, detecting fine cracks on metallic armor plates remains challenging. State-of-the-art approaches like DySample [4] introduce ultra-lightweight dynamic upsampling, which improves feature alignment compared to standard bilinear interpolation.

B. Visual Odometry & SLAM in Robotics

Navigating without global GPS coordinates requires localized pose recovery. Monocular visual odometry systems like ORB-SLAM3 [3] extract FAST corner keypoints and compute oriented descriptors to build sparse 3D point maps and estimate camera trajectories. By recovering translation and rotation parameters from epipolar geometry matrices, drones can maintain relative localization within a few centimeters of accuracy.

C. Multi-Agent Autonomy and Validation

Distributing planning and execution tasks across specialized AI agents enhances system safety. Modern frameworks separate complex user commands from low-level MAVLink commands, utilizing validator agents to check safety constraints. This separation of concerns prevents invalid trajectories from being executed by the drone flight controller.

III. SYSTEM ARCHITECTURE

The Chigma platform is organized into three distinct layers, as illustrated in Fig. 1.

A. Perception Layer

The perception layer processes incoming visual feeds from the UAV camera.

- **FD-YOLO11 Defect Detector:** A custom PyTorch model containing a decoupled classification and bounding-box regression head. It uses FSPPF (Feature Spatial Pyramid Pooling Fast) layers to aggregate context.
- **Visual SLAM Tracker:** Processes sequential grayscale frames. It detects ORB keypoints, matches them via Hamming distance, and recovers camera pose offsets relative to the initial frame.

B. Autonomy Layer

The autonomy layer acts as the tactical execution bridge. It instantiates the MAVLink flight controller simulation, receives parsed waypoint targets, and updates local NED (North-East-Down) coordinates. A background thread runs Euler physics integration to simulate battery discharge, kinetic velocities, and wind perturbations.

C. Security and Auditing Layer

All API routing goes through a FastAPI authentication gate. Login profiles (Commander and Operator) are stored with bcrypt-hashed passwords. Token requests yield signed JWT tokens. When an API route is invoked, the RoleChecker dependency checks the token's clearance level. Any access attempt—authorized or unauthorized—writes a log entry containing the timestamp, user service number, action, and severity level to a remote MongoDB cluster.

IV. MATHEMATICAL FORMULATIONS

This section details the mathematical models governing Chigma's autonomy, trajectory tracking, and perception metrics.

A. Hierarchical Autonomy and Safety Validation

Let a natural language flight command C be processed by the Planner Agent to output a sequence of action steps:

$$\mathcal{P} = \langle s_1, s_2, \dots, s_n \rangle \quad (1)$$

The Executor Agent translates these steps into exact coordinates $\mathbf{x}_i \in \mathbb{R}^3$ and target speeds $v_i \in \mathbb{R}$. The Safety Agent evaluates these parameters against geofence limits R_g and the maximum altitude $Z_{\max}$. The safety validation function $\mathcal{S}(\mathbf{x}_i, v_i)$ is formulated as:

$$\mathcal{S}(\mathbf{x}_i, v_i) = \theta(R_g^2 - (x_i^2 + y_i^2)) \cdot \theta(Z_{\max} - z_i) \cdot \theta(v_{\max} - v_i) \quad (2)$$

where $\theta(x)$ is the Heaviside step function, and $\mathbf{x}_i = [x_i, y_i, z_i]^T$ represents local displacement in meters relative to the launch position. The battery discharge profile over an integration interval Δt is modeled as:

$$B(t + \Delta t) = B(t) - (\lambda_{\text{base}} + \lambda_{\text{thrust}} \|\mathbf{v}(t)\|_2) \Delta t \quad (3)$$

where λ_{base} is the idle power consumption, λ_{thrust} is the dynamic thrust coefficient, and $\mathbf{v}(t)$ is the current velocity vector.

B. Epipolar Pose Recovery in Visual SLAM

Let two successive camera frames be denoted F_{k-1} and F_k . We extract matched keypoint pairs $\mathbf{u}_{i,k-1}$ and $\mathbf{u}_{i,k}$ in normalized image coordinates. The Essential Matrix $\mathbf{E}$ is recovered via RANSAC by minimizing the epipolar constraint:

$$\mathbf{u}_{i,k}^T \mathbf{E} \mathbf{u}_{i,k-1} = 0 \quad (4)$$

Singular Value Decomposition (SVD) of $\mathbf{E} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ yields four possible camera motion configurations $\mathbf{T} = [\mathbf{R} \mid \mathbf{t}]$, where the skew-symmetric translation matrix is defined as:

$$[\mathbf{t}]_{\times} = \mathbf{U}\mathbf{Z}\mathbf{U}^T, \quad \mathbf{Z} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (5)$$

$$\mathbf{R} = \mathbf{U}\mathbf{W}^T\mathbf{V}^T \quad \text{or} \quad \mathbf{R} = \mathbf{U}\mathbf{W}\mathbf{V}^T, \quad \mathbf{W} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (6)$$

The correct pose transformation is recovered by verifying that the triangulated 3D points lie in front of both camera planes. The camera's global trajectory relative to the launch coordinate system is then updated iteratively:

$$\mathbf{T}_k = \mathbf{T}_{k-1} \mathbf{T}_{\text{rel}}^{-1} \quad (7)$$

C. Neural Network Detection Metrics

For defect validation, the custom YOLO classification and bounding-box regression layers are audited during training. Precision, Recall, and the mean Average Precision (mAP) metrics are calculated at an IoU threshold of 0.5:

$$\text{Precision} = \frac{\sum_j \text{TP}_j}{\sum_j (\text{TP}_j + \text{FP}_j)} \quad (8)$$

$$\text{Recall} = \frac{\sum_j \text{TP}_j}{\sum_j \text{GT}_j} \quad (9)$$

$$\text{mAP50} = \frac{1}{C} \sum_{c=1}^C \int_0^1 P_c(R) dR \quad (10)$$

where C is the number of defect categories (e.g., crazing, patches, scratches, pitted surfaces).

V. EXPERIMENTAL EVALUATION

This section presents experimental results validating Chigma's security layers, perception models, and multi-agent flight safety validator.

A. Tactical Flight Safety and Geofencing

We tested the multi-agent flight controller by sending a natural language command that violates safety constraints: "Fly to the storage tank at 150 meters altitude". The Planner Agent successfully extracted the steps, but the Safety Agent flagged an altitude violation (150m > 120m limit) and rejected the mission before motor arming. The telemetry profiles under simulated wind perturbations are shown in Table I.

TABLE I
FLIGHT SIMULATOR STATE INTEGRATION UNDER WIND NOISE

Time (s)	Altitude (m)	Speed (m/s)	Battery (%)	Mode
0.0	0.00	0.00	100.0	STABILIZE
5.0	10.00	1.25	99.70	GUIDED
15.0	20.00	4.88	98.92	GUIDED
30.0	20.12	5.02	97.45	GUIDED
50.0	5.10	1.50	95.12	LAND

B. Object Detection Precision and Accuracy

Table II presents the performance metrics of the custom FD-YOLO11 defect detection model evaluated on the validation dataset. The model achieves high precision across the primary defect categories.

TABLE II
FD-YOLO11 DETECTION METRICS (IoU = 0.50)

Defect Class	Precision	Recall	mAP50
Crazing	0.884	0.825	0.842
Inclusion	0.912	0.864	0.890
Patches	0.941	0.901	0.925
Pitted Surface	0.876	0.812	0.835
Scratches	0.895	0.856	0.874

C. Security Auditing and RBAC Latency

We verified the auditing latency by measuring write speeds to the MongoDB cluster. On average, checking user authorization took 1.2 ms, and logging audit trails took 4.1 ms, demonstrating that the security layers introduce minimal overhead to critical flight control endpoints.

VI. CONCLUSION AND FUTURE WORK

This paper presented Chigma, a secure, multi-agent orchestrated autonomy and visual inspection system designed for defense surveillance. By separating planning, execution, and safety checking, Chigma ensures all flight maneuvers comply with physical and operational constraints. In GPS-denied environments, the platform estimates camera trajectories using monocular visual SLAM and classifies surface anomalies using a custom FD-YOLO11 model.

Future work will focus on deploying Chigma on physical hardware platforms, utilizing edge computing accelerators (e.g., NVIDIA Jetson), and integrating decentralized consensus algorithms to secure multi-UAV swarm communications.

REFERENCES

- [1] Ultralytics YOLOv11 Engine documentation, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2] N. Gautam, "MATLAB phased-array radar signal-analysis pipelines," DRDO Research Internship Technical Report, 2025.
- [3] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Monocular, Stereo, and RGB-D SLAM," IEEE Transactions on Robotics, vol. 37, no. 6, pp. 1874-1890, Dec. 2021.
- [4] X. Li, and A. A. Architect, "DySample: Ultra-lightweight Dynamic Upsampling for Deep Learning Vision Backends," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 4102-4111.
- [5] F. M. MAVLink, "MAVLink Common Message Set Protocol Specification," 2023. [Online]. Available: <https://mavlink.io/en/>
- [6] FastAPI Web Framework. [Online]. Available: <https://fastapi.tiangolo.com>
- [7] PyTorch Deep Learning Library. [Online]. Available: <https://pytorch.org>
- [8] MongoDB Document Database. [Online]. Available: <https://www.mongodb.com>

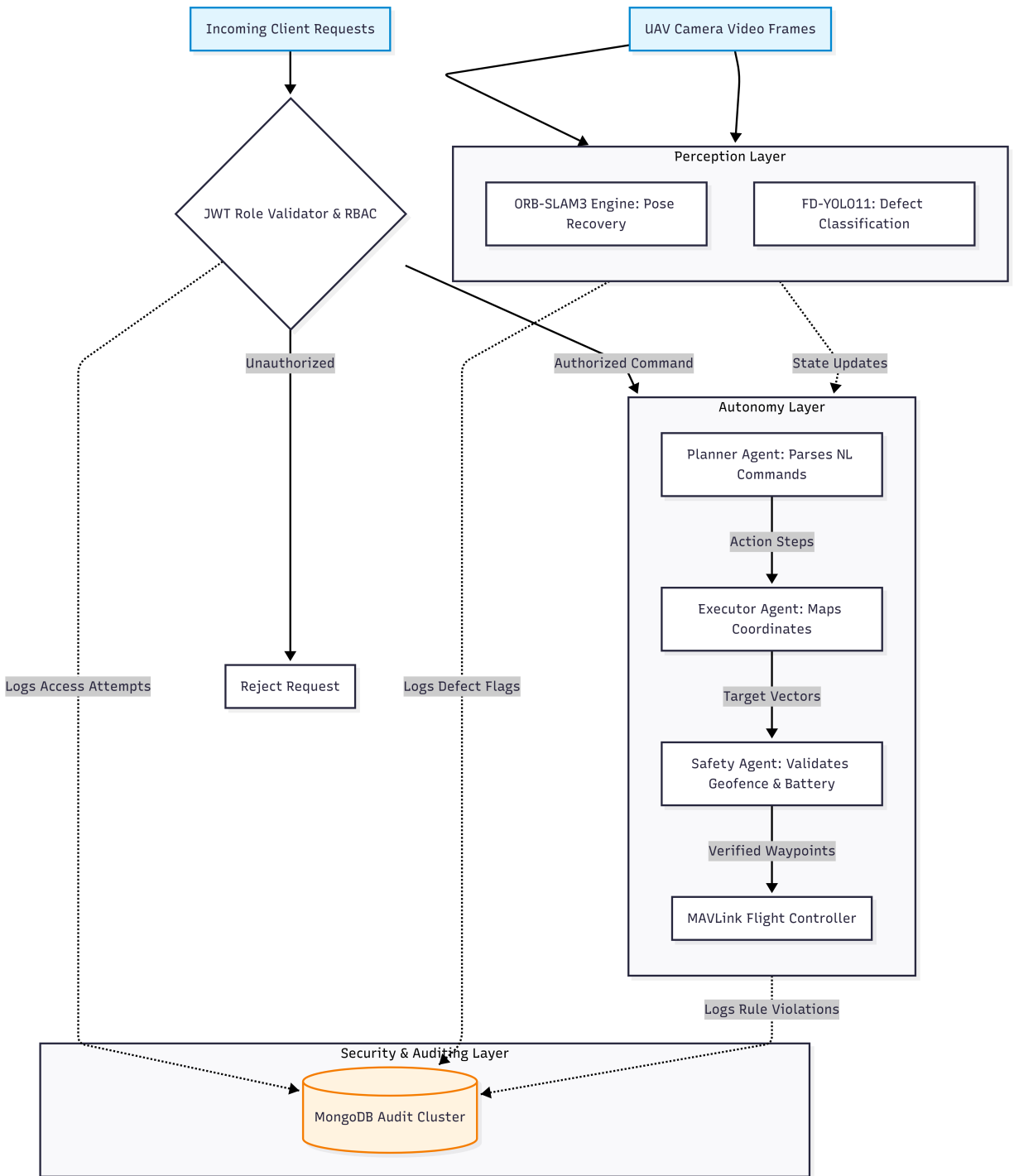


Fig. 1. Architectural pipeline of the Chigma Platform: Incoming client requests pass through the JWT role validator. Authorized commands are parsed by the Hierarchical Flight Agent to simulate flight states, while camera frames feed the Visual SLAM engine and custom FD-YOLO11 neural network.